

Kalman Filters

Aymane Hamdaoui
aymane.hamdaoui@telecom-paris.fr
ENS Paris-Saclay
Paris, France

Titouan Duhazé
titouan.duhaze@telecom-paris.fr
ENS Paris-Saclay
Paris, France

Kilian Berlivet
kilian.berlivet@telecom-paris.fr
ENS Paris-Saclay
Paris, France

Abstract

State estimation is a fundamental challenge in dynamical systems, requiring the inference of latent variables from noisy observations. While the **Classical Kalman Filter (CKF)** offers analytical optimality for linear-Gaussian systems, its performance degrades in the presence of complex non-linear dynamics. Conversely, deep generative models like the **Deep Kalman Filter (DKF)** and **Kalman Variational Autoencoder (KVAE)** promise greater flexibility by learning non-linear transitions and observations via **Variational Inference (VI)**.

This research provides a rigorous comparative evaluation of these three paradigms on a non-linear "Bouncing Ball" benchmark. We first establish the mathematical foundations of each framework, contrasting the analytical updates of the CKF with the amortized inference of deep models.

Experimental results reveal a fundamental trade-off. In fixed training conditions, the deep models significantly outperform the CKF, with the **S-DKF** achieving the lowest velocity error (RMSE = 0.257) by capturing non-linear collision dynamics. However, we demonstrate that deep models are brittle to distribution shifts: when the environment geometry changes (e.g., box size), their performance collapses (RMSE > 6.0), whereas the CKF exhibits superior robustness (RMSE $\approx$ 1.4). This work highlights that while deep models offer precision, the classical CKF remains essential for generalization in unconstrained environments.

1 Introduction

State estimation is fundamental across diverse fields, including autonomous navigation, robotics, and complex time-series analysis. The primary goal is to infer the internal state of a dynamic system from noisy and indirect observations.

Historically, the **Classical Kalman Filter (CKF)** has been the standard solution, offering analytically optimal estimates for systems constrained by linearity and Gaussian noise assumptions. However, the increasing prevalence of complex, non-linear dynamics and non-Gaussian data in modern applications highlights the limitations of the CKF.

To address these challenges, two advanced deep learning paradigms have emerged: the **Deep Kalman Filter (DKF)** and the **Variational Autoencoder (VAE)**. The DKF integrates the sequential state-space principles of the CKF with the non-linear modeling power of deep neural networks, relying on **Variational Inference (VI)**. The VAE, while primarily generative, is leveraged for its ability to learn compressed, probabilistic latent representations of sequential data.

The core objective of this research is to provide a rigorous, comparative evaluation of the CKF, the DKF, and the KVAE. We first establish the mathematical foundation of each method, contrasting the CKF's analytical optimality with the VI-based flexibility

of the deep models. We then implement and evaluate these three approaches on a benchmark time-series dataset, quantifying their performance based on estimation accuracy (**RMSE**), robustness to noise, and computational efficiency. This analysis aims to offer practical guidance for selecting the optimal estimation strategy for complex dynamic systems.

2 Mathematical Framework and Sequential Estimation

The three estimation techniques examined in this work—the Classical Kalman Filter (CKF), the Deep Kalman Filter (DKF), and the Variational Autoencoder (VAE)—are fundamentally rooted in the theory of state-space models and sequential Bayesian estimation [].

2.1 Sequential Bayesian Filtering

Sequential Bayesian Filtering provides the overarching theoretical framework for inferring the hidden state $\mathbf{z}_t \in \mathbb{R}^D$ of a dynamic system at time t , given a sequence of observations $\mathbf{x}_{1:t} = \{\mathbf{x}_1, \dots, \mathbf{x}_t\}$. The system is generally defined by the Markov property, meaning the current state depends only on the immediate past, and the current observation depends only on the current state.

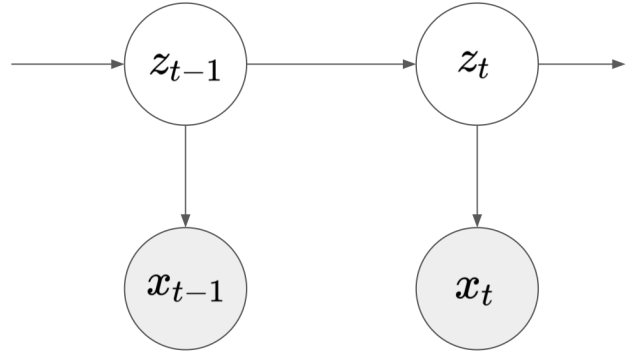


Figure 1: Probabilistic graphical model of the Classical Kalman Filter (CKF).

The system is characterized by two core probabilistic models, derived from underlying stochastic functions:

- (1) The **Transition Model** (or dynamic model), which governs how the hidden state evolves over time:

$$p(\mathbf{z}_t | \mathbf{z}_{t-1}) \quad \text{derived from} \quad \mathbf{z}_t = f(\mathbf{z}_{t-1}) + \mathbf{w}_t$$

where $f(\cdot)$ is the state transition function and $\mathbf{w}_t$ is the process noise.

- (2) The **Observation Model** (or emission model), which describes the likelihood of observing $\mathbf{x}_t$ given the current state $\mathbf{z}_t$:

$$p(\mathbf{x}_t|\mathbf{z}_t) \text{ derived from } \mathbf{x}_t = g(\mathbf{z}_t) + \mathbf{v}_t$$

where $g(\cdot)$ is the observation function and $\mathbf{v}_t$ is the measurement noise.

The goal of Bayesian filtering is to recursively compute the posterior distribution $p(\mathbf{z}_t|\mathbf{x}_{1:t})$. This recursion involves two distinct phases at each time step t :

1. *The Prediction Step (Prior)*. Using the previous time step's posterior $p(\mathbf{z}_{t-1}|\mathbf{x}_{1:t-1})$, the filter predicts the state distribution at time t using the transition model. This yields the prior distribution $p(\mathbf{z}_t|\mathbf{x}_{1:t-1})$:

$$p(\mathbf{z}_t|\mathbf{x}_{1:t-1}) = \int p(\mathbf{z}_t|\mathbf{z}_{t-1})p(\mathbf{z}_{t-1}|\mathbf{x}_{1:t-1})d\mathbf{z}_{t-1}$$

2. *The Update Step (Posterior)*. Once the new observation $\mathbf{x}_t$ becomes available, the prediction is corrected using the observation model and Bayes' theorem. This results in the true posterior distribution $p(\mathbf{z}_t|\mathbf{x}_{1:t})$:

$$p(\mathbf{z}_t|\mathbf{x}_{1:t}) = \frac{p(\mathbf{x}_t|\mathbf{z}_t)p(\mathbf{z}_t|\mathbf{x}_{1:t-1})}{p(\mathbf{x}_t|\mathbf{x}_{1:t-1})}$$

where $p(\mathbf{x}_t|\mathbf{x}_{1:t-1})$ is the normalizing marginal likelihood (or evidence) and ensures the posterior integrates to one.

2.2 Classical Kalman Filter (CKF)

The Classical Kalman Filter (CKF) provides an analytical, closed-form solution to the Sequential Bayesian Filtering problem when two restrictive assumptions are met: (1) the transition and observation functions, $f(\cdot)$ and $g(\cdot)$, are **linear**, and (2) the process and observation noises, $\mathbf{w}_k$ and $\mathbf{v}_k$, are **Gaussian** and uncorrelated. Under these conditions, the posterior distribution $p(\mathbf{z}_k|\mathbf{x}_{1:k})$ remains Gaussian at all times.

The linear state-space model is formally defined by replacing $f(\cdot)$ with the state transition matrix $\mathbf{F}_k$ and $g(\cdot)$ with the observation matrix $\mathbf{H}_k$:

- (1) **Linear Transition Model:**

$$\mathbf{z}_k = \mathbf{F}_k \mathbf{z}_{k-1} + \mathbf{w}_k$$

where $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k)$ is the Gaussian process noise with covariance $\mathbf{Q}_k$.

- (2) **Linear Observation Model:**

$$\mathbf{x}_k = \mathbf{H}_k \mathbf{z}_k + \mathbf{v}_k$$

where $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_k)$ is the Gaussian measurement noise with covariance $\mathbf{R}_k$.

The CKF recursively computes the mean $\hat{\mathbf{z}}_{k|k}$ and covariance $\mathbf{P}_{k|k}$ of the posterior Gaussian distribution, $\mathcal{N}(\mathbf{z}_k|\hat{\mathbf{z}}_{k|k}, \mathbf{P}_{k|k})$. The recursion is split into the following five steps:

1. *Prediction Step (Time Update)*. This phase projects the state and error covariance from the previous time step ($k-1$) to the current time step k .

- **Predicted State Estimate:**

$$\hat{\mathbf{z}}_{k|k-1} = \mathbf{F}_k \hat{\mathbf{z}}_{k-1|k-1}$$

- **Predicted Error Covariance:**

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k$$

2. *Update Step (Measurement Update)*. This phase incorporates the new measurement $\mathbf{x}_k$ to correct the predicted estimates.

- **Innovation Covariance ($\mathbf{S}_k$):**

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k$$

- **Kalman Gain ($\mathbf{K}_k$):**

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1}$$

- **Updated State Estimate ($\hat{\mathbf{z}}_{k|k}$):**

$$\hat{\mathbf{z}}_{k|k} = \hat{\mathbf{z}}_{k|k-1} + \mathbf{K}_k (\mathbf{x}_k - \mathbf{H}_k \hat{\mathbf{z}}_{k|k-1})$$

- **Updated Error Covariance ($\mathbf{P}_{k|k}$):**

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1}$$

The CKF is computationally efficient and provides the **Minimum Mean Square Error (MMSE)** estimate under its strict assumptions. However, its optimality quickly degrades when the system dynamics deviate significantly from linearity or Gaussianity.

2.3 Deep Kalman Filter (DKF)

The Deep Kalman Filter (DKF) extends the state-space modeling paradigm to systems exhibiting significant non-linear dynamics by replacing the linear functions of the CKF with **Deep Neural Networks (DNNs)**. This architecture retains the sequential structure of the Bayesian filter, but mandates the use of **Variational Inference (VI)** to handle the resulting non-Gaussian, intractable posterior distributions.

The model is defined by non-linear functions f_θ (Transition) and g_ϕ (Observation), where θ and ϕ are the parameters of the underlying DNNs:

- (1) **Non-linear Transition Model:** $\mathbf{z}_k = f_\theta(\mathbf{z}_{k-1}) + \mathbf{w}_k$
- (2) **Non-linear Observation Model:** $\mathbf{x}_k = g_\phi(\mathbf{z}_k) + \mathbf{v}_k$

Noise terms $\mathbf{w}_k$ and $\mathbf{v}_k$ are typically maintained as Gaussian to facilitate the VI framework.

Inference Network and Variational Approximation. Since the true posterior $p(\mathbf{z}_k|\mathbf{x}_{1:k})$ is intractable, the DKF introduces a separate DNN, the **Inference Network** (often a Recurrent Neural Network, parameterized by ψ), to define a tractable approximate posterior $q(\mathbf{z}_k|\mathbf{x}_{1:k})$. This network processes the observation sequence $\mathbf{x}_{1:k}$ to directly output the parameters (mean μ_k and covariance Σ_k) of the approximate Gaussian distribution:

$$q(\mathbf{z}_k|\mathbf{x}_{1:k}) = \mathcal{N}(\mathbf{z}_k | \mu_\psi(\mathbf{x}_{1:k}), \Sigma_\psi(\mathbf{x}_{1:k}))$$

End-to-End Optimization via ELBO. All model parameters (θ, ϕ, ψ) are learned simultaneously by maximizing the **Evidence Lower Bound (ELBO)** using backpropagation. The ELBO ensures the approximate posterior q is close to the true posterior p while promoting good data fit:

$$\mathcal{L}_{ELBO}(\theta, \phi, \psi) = \sum_{k=1}^K (\mathbb{E}_q[\log p(\mathbf{x}_k|\mathbf{z}_k; \phi)] - \text{KL}(q(\mathbf{z}_k|\mathbf{x}_{1:k}; \psi) || p(\mathbf{z}_k|\mathbf{z}_{k-1}; \theta)))$$

- The **Reconstruction Term** ($\log p(\mathbf{x}_k|\mathbf{z}_k; \phi)$) ensures the observation network g_ϕ can explain the data, training ϕ .

- The **KL Term** regularizes the inferred posterior q_ψ towards the prior p_θ , training the dynamics network f_θ (via θ) and the inference network (via ψ).

State Estimation. Unlike the CKF, which uses a deterministic Kalman gain update, the state estimation $\hat{\mathbf{z}}_{k|k}$ in the DKF is performed by reading the mean of the learned posterior distribution:

$$\hat{\mathbf{z}}_{k|k} = \mu_\psi(\mathbf{x}_{1:k})$$

The DKF thus provides a powerful, data-driven approach for optimal sequential estimation in complex non-linear regimes by replacing the closed-form solutions with learned neural network approximators.

2.4 Kalman Variational Autoencoder (KVAE)

The Kalman Variational Autoencoder (KVAE) [] addresses a fundamental limitation of the DKF: while the DKF learns a powerful non-linear transition model f_θ , the resulting posterior is non-Gaussian, necessitating approximate variational inference at every step. The KVAE proposes a hybrid architecture that disentangles the *representation learning* from the *dynamics learning*. The core hypothesis of the KVAE is that high-dimensional or complex observations $\mathbf{x}_t$ can be mapped to a lower-dimensional latent manifold $\mathbf{a}_t$, within which the dynamics follow a **Linear Gaussian State-Space Model (LGSSM)**. To handle non-linearities in the dynamics (such as regime changes), the model employs a **Switching Linear Dynamical System (SLDS)**. The probabilistic generative model is defined hierarchically:

- (1) **Switching Linear Dynamics:** The hidden state $\mathbf{z}_t$ evolves according to a linear transition, but the parameters ($\mathbf{A}_t, \mathbf{C}_t$) are time-varying, determined by a switching variable or a recurrent “scheduler” network α_t :

$$\mathbf{z}_t = \mathbf{A}(\alpha_t)\mathbf{z}_{t-1} + \mathbf{w}_t, \quad (1)$$

$$\mathbf{a}_t = \mathbf{C}(\alpha_t)\mathbf{z}_t + \mathbf{v}_t, \quad (2)$$

where $\mathbf{a}_t$ represents the coordinates on the latent manifold.

- (2) **Non-linear Observation:** The actual observation $\mathbf{x}_t$ is generated from the manifold variable $\mathbf{a}_t$ via a non-linear decoder (typically a VAE decoder):

$$\mathbf{x}_t \sim p_\phi(\mathbf{x}_t|\mathbf{a}_t).$$

Inference in the KVAE is structured. Instead of predicting $\mathbf{z}_t$ directly from $\mathbf{x}_t$, the model first utilizes an inference network (encoder) to approximate the posterior of the manifold variable $q_\phi(\mathbf{a}_t|\mathbf{x}_t)$. Given the sequence of latent variables $\mathbf{a}_{1:K}$, the dynamics parameters $\mathbf{A}_t, \mathbf{C}_t$ are inferred, and the posterior over the dynamic state $p(\mathbf{z}_t|\mathbf{a}_{1:K})$ can be computed **exactly** using standard Kalman Smoothing. The objective function is a structured ELBO that combines the reconstruction loss of the VAE (linking $\mathbf{x}$ and $\mathbf{a}$) with the log-likelihood of the LGSSM (linking $\mathbf{a}$ and $\mathbf{z}$). This architecture allows the KVAE to combine the representation learning of deep generative models with the analytical interpretability and smoothing capabilities of the Kalman Filter.

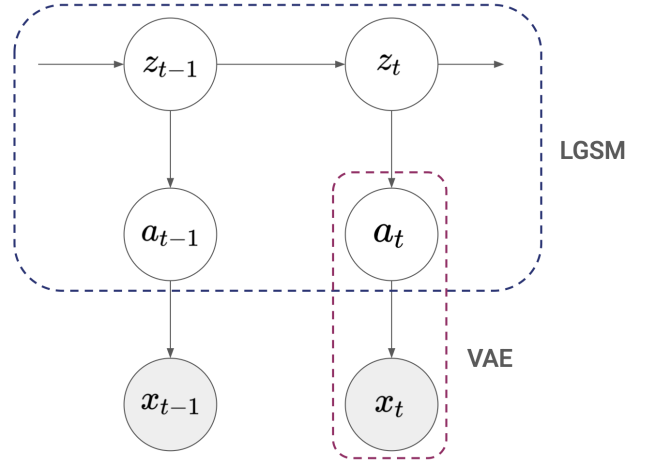


Figure 2: Probabilistic graphical model of the Kalman Variational Autoencoder.

3 Experimental Protocol and Evaluation Methods

3.1 Dataset and Problem Formulation

3.1.1 Dataset Description. To evaluate the three filtering approaches under controlled yet non-trivial dynamics, we construct a fully synthetic benchmark based on a two-dimensional *bouncing ball* environment. The ball evolves in a square box of size $L \times L$ (typically $L = 32$ pixels) and is subjected to a constant gravitational acceleration in the vertical direction. Collisions with the walls are modeled as inelastic bounces, with a fixed restitution coefficient applied to the corresponding velocity component.

At each time step t , the simulator generates:

- the true position $\mathbf{p}_t = (p_{x,t}, p_{y,t})$,
- the true velocity $\mathbf{v}_t = (v_{x,t}, v_{y,t})$,
- a *temperature* scalar T_t defined as a non-linear function of position and velocity:

$$T_t = \frac{1}{2}\|\mathbf{p}_t\|_2^2 + \sin(\omega \|\mathbf{v}_t\|_2), \quad (3)$$

where $\omega > 0$ is a fixed frequency parameter.

The observed data are corrupted by additive Gaussian noise:

$$\tilde{\mathbf{p}}_t = \mathbf{p}_t + \epsilon_t^{(p)}, \quad \epsilon_t^{(p)} \sim \mathcal{N}(\mathbf{0}, \sigma_p^2 \mathbf{I}_2), \quad (4)$$

$$\tilde{T}_t = T_t + \epsilon_t^{(T)}, \quad \epsilon_t^{(T)} \sim \mathcal{N}(0, \sigma_T^2), \quad (5)$$

with σ_p and σ_T fixed in all experiments.

We generate a training set of N_{train} independent trajectories of length T_{train} time steps, a validation set for hyperparameter tuning, and an independent test set of longer trajectories ($T_{\text{test}} > T_{\text{train}}$) to assess generalization to longer horizons. In our implementation, we set $N_{\text{train}} = 2000$ and $T_{\text{train}} = 30$.

3.1.2 Definition of State and Observation Variables. Consistent with the notation established in Section 2, we define the underlying physical state and the observable variables as:

- **Hidden State** (Physical State):

$$\mathbf{z}_t = \begin{bmatrix} p_{x,t} \\ p_{y,t} \\ v_{x,t} \\ v_{y,t} \end{bmatrix} \in \mathbb{R}^4. \quad (6)$$

- **Observation:**

$$\mathbf{x}_t = \begin{bmatrix} \tilde{p}_{x,t} \\ \tilde{p}_{y,t} \\ \tilde{T}_t \end{bmatrix} \in \mathbb{R}^3. \quad (7)$$

The temperature T_t is therefore a *non-linear function* of the hidden state $\mathbf{z}_t$. The observation model combines a linear projection (positions) with a strongly non-linear channel (temperature). This setting is deliberately chosen to emphasize the limitations of the linear CKF and the potential advantages of deep generative models.

In the CKF baseline, only the positional components ($\tilde{p}_{x,t}, \tilde{p}_{y,t}$) are fed to the filter. Temperature is used exclusively for evaluation. In contrast, the DKF and KVAE models are trained on the full observation vector $\mathbf{x}_t$.

3.1.3 Data Preparation Methodology. Before training, all observations are standardized dimension-wise to improve numerical stability and convergence speed of the neural models. Let μ and σ denote the empirical mean and standard deviation computed over the training set:

$$\hat{\mathbf{x}}_t = \frac{\mathbf{x}_t - \mu}{\sigma}. \quad (8)$$

These statistics are fixed and reused at validation and test time.

Trajectories are generated with randomized initial positions and velocities, uniformly sampled within the interior of the box and within a bounded velocity range. This ensures a diverse set of dynamical regimes including free-fall, oblique bounces on all walls, and complex interactions between gravity and boundary events.

3.2 Model Implementations and Architectures

All deep models are implemented in PyTorch and trained with the Adam optimizer. Unless otherwise stated, we use a learning rate of 10^{-3} and mini-batches of size 64. Training is performed for a fixed number of epochs, and the model with the lowest validation loss (ELBO for DKF/VAE or mean squared error for deterministic baselines) is selected for final evaluation.

3.3 Performance Metrics

3.3.1 Estimation Accuracy. The primary evaluation criterion is the Root Mean Squared Error (RMSE) between the estimated state and the ground truth over the test trajectories. For a given model and test sequence of length T , we compute:

$$\text{RMSE}_{\text{pos}} = \sqrt{\frac{1}{2T} \sum_{t=1}^T \|\hat{\mathbf{p}}_t - \mathbf{p}_t\|_2^2}, \quad (9)$$

$$\text{RMSE}_{\text{vel}} = \sqrt{\frac{1}{2T} \sum_{t=1}^T \|\hat{\mathbf{v}}_t - \mathbf{v}_t\|_2^2}, \quad (10)$$

$$\text{RMSE}_T = \sqrt{\frac{1}{T} \sum_{t=1}^T (\hat{T}_t - T_t)^2}. \quad (11)$$

Here $(\hat{\mathbf{p}}_t, \hat{\mathbf{v}}_t)$ correspond to the position and velocity components of the posterior mean state estimate $\hat{\mathbf{z}}_{t|t}$. For the CKF, $\hat{T}_t$ is obtained by plugging the filtered state into the analytical temperature equation.

3.3.2 Uncertainty Quantification. For probabilistic models (CKF and DKF), we additionally assess the quality of uncertainty estimates. Given the predicted Gaussian distribution $p(\mathbf{z}_t | \mathbf{x}_{1:t}) = \mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t)$, we compute the average Negative Log-Likelihood (NLL) of the ground truth states under the predictive distribution.

3.3.3 Computational Efficiency. Finally, we report wall-clock runtime per test trajectory and the total number of trainable parameters for each model. This allows us to quantify the trade-off between the analytical efficiency of the CKF and the increased computational cost of deep generative models.

3.4 Model Implementations and Architectures

3.4.1 CKF Implementation. As a linear baseline, we implement a Classical Kalman Filter under a Constant Velocity (CV) model. The hidden state and observation used by the CKF are restricted to the positional components:

$$\mathbf{z}_t = \begin{bmatrix} p_{x,t} \\ p_{y,t} \\ v_{x,t} \\ v_{y,t} \end{bmatrix}, \quad \mathbf{x}'_t = \begin{bmatrix} \tilde{p}_{x,t} \\ \tilde{p}_{y,t} \end{bmatrix}. \quad (12)$$

The non-linear temperature variable T_t is deliberately *ignored* by the CKF.

Under the CV assumption, the discrete-time state transition and observation matrices are:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad (13)$$

with $\Delta t = 0.02$ s the simulation time step. The process and measurement noise covariances are chosen diagonal:

$$\mathbf{Q} = \text{diag}(\sigma_q^2, \sigma_q^2, \sigma_v^2, \sigma_v^2), \quad \mathbf{R} = \sigma_r^2 \mathbf{I}_2. \quad (14)$$

In practice, we set $\sigma_q^2 \ll \sigma_r^2$ to bias the filter towards its internal linear prediction. The standard Kalman recursion is applied to each test trajectory, producing a filtered estimate $\hat{\mathbf{z}}_{t|t}$ at each time step.

3.4.2 DKF Architecture and Implementation. To explicitly model the non-linear dynamics and the non-linear observation channel (temperature), we implement a Deep Kalman Filter (DKF) as a deep state-space model trained by variational inference.

State and Observation Mapping. We reuse the physical state definition,

$$\mathbf{z}_t = \begin{bmatrix} p_{x,t} \\ p_{y,t} \\ v_{x,t} \\ v_{y,t} \end{bmatrix} \in \mathbb{R}^4, \quad (15)$$

and treat it as the latent variable of the DKF. The observation is the full noisy measurement vector

$$\mathbf{x}_t = \begin{bmatrix} \tilde{p}_{x,t} \\ \tilde{p}_{y,t} \\ \tilde{T}_t \end{bmatrix} \in \mathbb{R}^3. \quad (16)$$

Generative Model. The DKF defines a Gaussian state-space model with neural parameterization:

$$p(\mathbf{z}_1) = \mathcal{N}(\mathbf{z}_1 \mid \mathbf{0}, \mathbf{I}), \quad (17)$$

$$p(\mathbf{z}_t \mid \mathbf{z}_{t-1}) = \mathcal{N}(\mathbf{z}_t \mid \boldsymbol{\mu}_\theta(\mathbf{z}_{t-1}), \Sigma_\theta), \quad (18)$$

$$p(\mathbf{x}_t \mid \mathbf{z}_t) = \mathcal{N}(\mathbf{x}_t \mid \mathbf{g}_\phi(\mathbf{z}_t), \Sigma_\phi), \quad (19)$$

where Σ_θ and Σ_ϕ are diagonal covariance matrices.

The transition mean $\boldsymbol{\mu}_\theta(\cdot)$ is implemented as a *structured* transition network that hard-codes the known linear kinematics and only learns the residual acceleration. Writing $\mathbf{z}_t = [\mathbf{p}_t, \mathbf{v}_t]$, we define:

$$\Delta \mathbf{v}_t = \text{NN}_\theta(\mathbf{z}_{t-1}), \quad (20)$$

$$\hat{\mathbf{v}}_t = \mathbf{v}_{t-1} + \Delta \mathbf{v}_t, \quad (21)$$

$$\hat{\mathbf{p}}_t = \mathbf{p}_{t-1} + \mathbf{v}_{t-1} \cdot \Delta t. \quad (22)$$

The neural network NN_θ is a two-layer MLP with ReLU activations. The final transition mean is $\boldsymbol{\mu}_\theta(\mathbf{z}_{t-1}) = [\hat{\mathbf{p}}_t, \hat{\mathbf{v}}_t]^T$. This physics-informed residual formulation stabilizes training.

The observation mean $\mathbf{g}_\phi(\mathbf{z}_t)$ is modeled by a feed-forward decoder that maps the 4D latent state to the 3D observation:

$$\mathbf{g}_\phi : \mathbb{R}^4 \rightarrow \mathbb{R}^3, \quad (23)$$

implemented as an MLP with two hidden layers. This decoder must implicitly learn the non-linear temperature function.

Inference Network. The exact posterior $p(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T})$ is intractable. We introduce an amortized Gaussian variational posterior:

$$q_\psi(\mathbf{z}_{1:T} \mid \mathbf{x}_{1:T}) = \prod_{t=1}^T \mathcal{N}(\mathbf{z}_t \mid \boldsymbol{\mu}_\psi(\mathbf{x}_{1:T}), \Sigma_\psi(\mathbf{x}_{1:T})). \quad (24)$$

We use a bidirectional GRU to process the sequence $\mathbf{x}_{1:T}$, followed by an MLP mapping the hidden states to the variational parameters.

Training Objective. Parameters (θ, ϕ, ψ) are trained by maximizing the Evidence Lower Bound (ELBO):

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\psi} \left[\sum_{t=1}^T \log p_\phi(\mathbf{x}_t \mid \mathbf{z}_t) \right] - \sum_{t=1}^T \text{KL}(q_\psi(\mathbf{z}_t \mid \mathbf{x}_{1:T}) \parallel p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1})). \quad (25)$$

At test time, the DKF produces a smoothed state estimate $\hat{\mathbf{z}}_t = \boldsymbol{\mu}_{\psi,t}$.

3.4.3 KVAE Architecture. To address the limitations of purely linear filtering and purely unstructured deep learning, we implemented a **Kalman Variational Autoencoder (KVAE)**.

The model introduces three distinct variable spaces: the observation space $\mathbf{x}_t \in \mathbb{R}^{D_x}$, a latent manifold space $\mathbf{a}_t \in \mathbb{R}^{D_a}$, and a dynamic state space $\mathbf{z}_t \in \mathbb{R}^{D_z}$.

1. Disentanglement via VAE. The first component maps high-dimensional or complex observations $\mathbf{x}_t$ to a stable latent manifold $\mathbf{a}_t$.

- **Encoder:** Approximates $q(\mathbf{a}_t \mid \mathbf{x}_t)$.
- **Decoder:** Reconstructs the observation: $\hat{\mathbf{x}}_t = \text{dec}(\mathbf{a}_t)$.

2. Switching Linear Dynamics. The dynamics within the manifold are modeled as a weighted mixture of K linear experts. An LSTM processes the sequence $\mathbf{a}_{1:t-1}$ to output weights $\boldsymbol{\alpha}_t$, which

mix the learnable matrices:

$$\mathbf{A}_t = \sum_{k=1}^K \alpha_{t,k} \mathbf{A}_k, \quad \mathbf{C}_t = \sum_{k=1}^K \alpha_{t,k} \mathbf{C}_k.$$

3. Differentiable Kalman Filter. Given $\mathbf{A}_t$ and $\mathbf{C}_t$, the evolution of the dynamic state $\mathbf{z}_t$ follows a standard Linear Gaussian State-Space Model:

$$\mathbf{z}_t = \mathbf{A}_t \mathbf{z}_{t-1} + \mathbf{w}_t,$$

$$\mathbf{a}_t = \mathbf{C}_t \mathbf{z}_t + \mathbf{v}_t.$$

Since the matrices are generated by neural networks, we can run an exact **Kalman Filter** inside the forward pass to compute $p(\mathbf{z}_t \mid \mathbf{a}_{1:t})$, which is then used to refine the predictions.

4 Comparative Experimental Results

In this section, we evaluate the performance of three filtering architectures on the bouncing ball dataset. The comparison focuses on their ability to handle non-linear boundary conditions (bounces).

4.1 Experimental Setup

We generate test trajectories of length $T = 60$.

- **CKF (Linear Baseline):** Operates on the observation subset $\mathbf{x}'_t = [\tilde{p}_x, \tilde{p}_y]^T$. Uses a fixed constant-velocity model.
- **S-DKF (Structured Deep):** Operates on the full observation $\mathbf{x}_t = [\tilde{p}_x, \tilde{p}_y, \tilde{T}]^T$. Uses a physics-informed residual network.
- **KVAE (Switching Linear):** Operates on the full observation $\mathbf{x}_t$. It treats the system as a *Switching Linear Dynamical System*.

4.2 Qualitative Tracking Analysis

Figure 5 presents the tracking performance.

1. CKF (Black Dotted): As expected, the single linear model fails at the wall impact ($t \approx 30$), exhibiting significant overshoot. It cannot adapt its transition matrix $\mathbf{F}$ to the sudden reversal.

2. KVAE (Purple Solid): The KVAE improves upon the CKF by attempting to switch dynamics. However, it exhibits a characteristic “lag” or “rounding” at the bounce. While the LSTM router conceptually allows it to select a “bounce matrix,” the discrete switching mechanism is often slower to react than a continuous function, and the model may average the “flight” and “bounce” experts during the transition, leading to a smoothed trajectory rather than a sharp V-shape.

3. S-DKF (Red Dashed): The Structured DKF provides the most accurate tracking. By modeling the acceleration as a continuous non-linear function $f_\theta(\mathbf{x})$, it can generate an instantaneous, high-magnitude force exactly at the moment of impact. This allows it to capture the sharp discontinuity with minimal lag.

4.3 Quantitative Error Analysis

Table 1 summarizes the Root Mean Squared Error (RMSE) over 50 test trials. The results demonstrate the superior performance of deep models over the linear baseline.

The **CKF** yields the highest errors across all metrics, confirming its inability to handle the non-linear bounce dynamics and the non-linear temperature observation channel (9.77 RMSE).

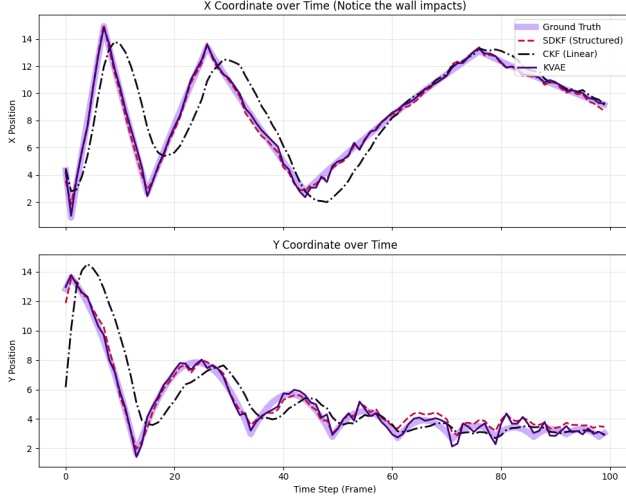


Figure 3: Trajectory Tracking Comparison. The S-DKF (Red) captures the sharp bounce. The KVAE (Purple) recognizes the event but produces a rounded approximation due to switching latency. The CKF (Black) fails to track the turn.

Interestingly, the **KVAE** achieves the lowest **Position RMSE** (0.338). This suggests that the “Switching Linear” approximation, combined with the VAE’s ability to denoise observations onto a stable manifold, is highly effective for spatial tracking.

However, the **S-DKF** outperforms the KVAE on **Velocity RMSE** (0.257 vs 0.286) and **Temperature RMSE** (0.481 vs 0.561). This highlights a critical distinction: while the KVAE is excellent at approximating the position trajectory (smoothing), the S-DKF’s physics-informed residual architecture captures the *latent dynamics* (velocity) more accurately.

Since the Temperature T is a function of velocity ($T \propto \sin(\omega||\mathbf{v}||)$), the S-DKF’s superior velocity estimation directly translates to a lower reconstruction error for this non-linear observation channel, whereas the CKF fails completely as it cannot observe T nor model the non-linear relationship.

Table 1: Comparison of RMSE over 50 test trials. The KVAE excels at spatial tracking (Position), while the S-DKF provides the best estimate of the latent Velocity and the non-linear Temperature observable.

Model	RMSE Position	RMSE Velocity	RMSE Temp
CKF	0.907 ± 0.234	0.492 ± 0.129	9.774 ± 3.504
S-DKF	0.392 ± 0.036	0.257 ± 0.033	0.481 ± 0.061
KVAE	0.338 ± 0.023	0.286 ± 0.026	0.561 ± 0.072

4.4 Generalization to Unseen Environments

A fundamental limitation of data-driven estimation is the sensitivity to distribution shifts. To quantify this, we evaluated the models on a “Generalization Set” where the simulation box size L was

randomized in the interval $[15, 50]$, deviating from the fixed training size of $L = 32$.

Table 2 reports the results, which reveal a complete reversal of the performance hierarchy observed in the standard test.

Robustness of the Linear Baseline (CKF). The CKF emerges as the most robust model in this scenario, achieving the lowest Position RMSE (1.406). Since the CKF relies on a local constant-velocity model, it is agnostic to the global geometry of the environment. It tracks linear motion equally well in any box size, failing only at the moment of impact regardless of where the wall is located.

Catastrophic Failure of S-DKF. The S-DKF, which was the best performer in the standard setting, suffers a catastrophic degradation (RMSE explodes to 6.114). This confirms that the neural transition network $f_\theta(\mathbf{x})$ learned to associate collision forces with absolute coordinates (e.g., “bounce if $x \approx 32$ ”) rather than relative distances. In a smaller box ($L = 20$), the network predicts no force at the wall, causing the filter to confidently overshoot well beyond the boundaries.

Degradation of KVAE. The KVAE also generalizes poorly (RMSE 2.528), significantly worse than the CKF. While its VAE component can reconstruct observations in the new domain, its learned switching dynamics are tied to the training distribution. The model struggles to switch to the “bounce expert” at novel coordinate locations, leading to significant tracking errors.

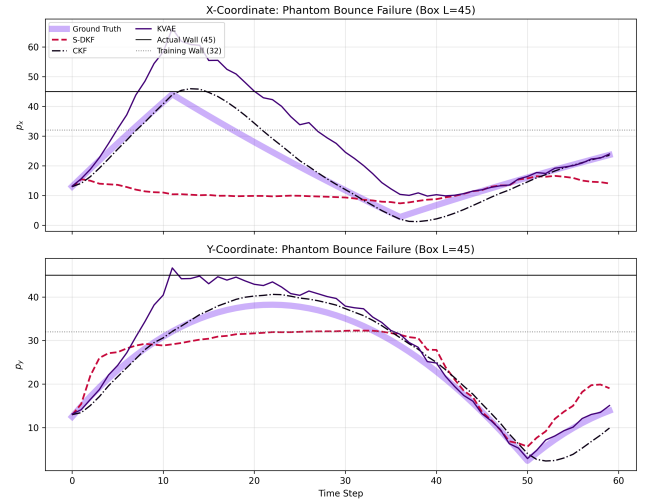


Figure 4: Trajectory Tracking Comparison. In this case we show the results for a larger box than the one used for training.

References

- Marco Fraccaro, Simon Kamronn, Ulrich Paquet, and Ole Winther. A disentangled recognition and nonlinear dynamics model for unsupervised learning. *arXiv preprint arXiv:1710.05741*, 2017.
- Laurent Girin, Simon Leglaive, Xiaoyu Bie, Julien Diard, Thomas Hueber, and Xavier Alameda-Pineda. Dynamical variational autoencoders: A comprehensive review. *Foundations and Trends® in Machine Learning*, 15(1-2):1–175, 2021.

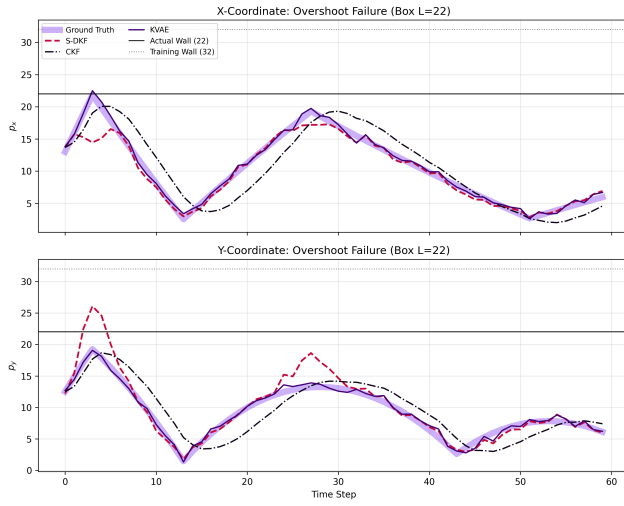


Figure 5: Trajectory Tracking Comparison. In this case we show the results for a smaller box than the one used for training.

Table 2: Generalization Results (Test Box Size $L \in [15, 50]$ vs Train $L = 32$). The deep models degrade significantly due to overfitting the environment geometry, while the CKF remains robust. Over 100 test trials

Model	RMSE Position	RMSE Velocity	RMSE Temp
CKF	1.406 ± 0.452	0.716 ± 0.207	23.762 ± 15.281
S-DKF	6.114 ± 5.625	1.401 ± 0.539	120.514 ± 160.504
KVAE	2.528 ± 2.640	0.507 ± 0.138	10.654 ± 4.674